Rotary Position Embedding (RoPE)

Explicación paso a paso

Deep Learning — EAFIT

2026

1. La idea central

Queremos que el producto punto entre un query $\mathbf{q}$ en la posición m y una key $\mathbf{k}$ en la posición n dependa únicamente de **la distancia entre posiciones**, no de sus posiciones absolutas:

$$\mathbf{q}_m^\top \mathbf{k}_n = f(m-n)$$

Esto permite que el modelo generalice a secuencias más largas que las vistas en entrenamiento.

---

2. Rotación en 2D

Toma un vector de dos dimensiones $\mathbf{x} = (x_1, x_2)$. RoPE lo rota un ángulo $m\theta$ según la posición m:

$$\mathbf{x}_m = R(m\theta)\,\mathbf{x} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

Expandiendo:

$$x_1' = x_1 \cos m\theta - x_2 \sin m\theta$$
$$x_2' = x_1 \sin m\theta + x_2 \cos m\theta$$

¿Por qué rota? Cualquier vector 2D se puede escribir en coordenadas polares como $x_1 = r\cos\phi$, $x_2 = r\sin\phi$. Usando las identidades de suma de ángulos:

$$x_1' = r\cos(\phi + m\theta), \qquad x_2' = r\sin(\phi + m\theta)$$

El resultado es el mismo vector con ángulo $\phi + m\theta$ — rotado exactamente $m\theta$, con magnitud r intacta.

---

3. Por qué esto da posición relativa

El producto punto entre $\mathbf{q}$ rotado en m y $\mathbf{k}$ rotado en n:

$$\mathbf{q}_m^\top \mathbf{k}_n = \mathbf{q}^\top R(m\theta)^\top R(n\theta)\,\mathbf{k}$$

Las matrices de rotación son ortogonales, por lo tanto $R(\alpha)^\top = R(-\alpha)$:

$$= \mathbf{q}^\top R((n-m)\theta)\,\mathbf{k}$$

El resultado depende solo de $n - m$. ✓

---

4. Escalando a dimensión d

Un vector real tiene d dimensiones. Se divide en $d/2$ **pares consecutivos**:

$$(x_1, x_2),\ (x_3, x_4),\ \ldots,\ (x_{d-1}, x_d)$$

Cada par i se rota con su propia frecuencia θ_i:

$$\theta_i = \frac{1}{10000^{\,2i/d}}, \qquad i = 0, 1, \ldots, \frac{d}{2} - 1$$

La matriz de rotación completa es **block-diagonal**:

$$\mathbf{R}_m = \begin{pmatrix} R(m\theta_0) & & \\ & \ddots & \\ & & R(m\theta_{d/2-1}) \end{pmatrix}$$

¿Qué significa cada θ_i? (ejemplo con $d = 8$)

par i	dimensiones	$\boldsymbol{\theta_i}$	período (tokens)
0	(x_1, x_2)	1.0	≈ 6
1	(x_3, x_4)	0.178	≈ 35
2	(x_5, x_6)	0.010	≈ 628
3	(x_7, x_8)	0.0018	$\approx 3{,}500$

- **Par 0** $\rightarrow$ rota rápido: captura dependencias locales (tokens cercanos).
- **Par** $d/2 - 1$ $\rightarrow$ rota lento: captura estructura global (tokens lejanos).

Esto es análogo al encoding sinusoidal original del Transformer, pero aplicado como **rotación multiplicativa** en lugar de suma aditiva.

---

5. El truco con números complejos

Rotar un par (x_1, x_2) es equivalente a multiplicar el número complejo $z = x_1 + ix_2$ por el fasor $e^{im\theta_i}$:

$$z \cdot e^{im\theta} = (x_1 + ix_2)(\cos m\theta + i \sin m\theta)$$

Esta es exactamente la operación que hace `torch.polar` seguido de multiplicación compleja:

```
import torch

def precompute_freqs_cis(dim: int, seq_len: int, theta: float = 10000.0):
    # \theta{}_i = 1 / (10000 ^ (2i/dim)), para i = 0..dim//2-1
    freqs = 1.0 / (theta ** (torch.arange(0, dim, 2).float() / dim))
    positions = torch.arange(seq_len).float()
    freqs = torch.outer(positions, freqs)        # (seq_len, dim//2)
    return torch.polar(torch.ones_like(freqs), freqs)  # e^{i\theta{}}, complejo

def apply_rope(x: torch.Tensor, freqs_cis: torch.Tensor) -> torch.Tensor:
    # x: (batch, seq_len, n_heads, head_dim)
    x_complex = torch.view_as_complex(
        x.float().reshape(*x.shape[:-1], -1, 2)
    )
    freqs_cis = freqs_cis.unsqueeze(0).unsqueeze(2)
    x_rotated = torch.view_as_real(x_complex * freqs_cis).flatten(-2)
    return x_rotated.type_as(x)
```

---

6. Dónde ocurre la rotación

Perspectiva	Cuándo ocurre	Dónde se usa
Derivación matemática	Dentro del inner product	Paper original (Su et al., 2021)
Implementación práctica	Pre-rotación de **q** y **k**	LLaMA, Mistral, Gemma ...

Matemáticamente son idénticos porque la rotación es lineal. Pre-rotar antes de attention es más limpio:

```
# Dentro del forward de attention:
q = apply_rope(q, freqs_cis)
k = apply_rope(k, freqs_cis)

# Luego attention normal:
scores = q @ k.transpose(-2, -1) / math.sqrt(head_dim)
attn   = torch.softmax(scores, dim=-1)
```

Resumen

La propiedad clave de RoPE es que el producto punto preserva la posición relativa:

$$\langle \mathrm{RoPE}(\mathbf{q}, m),\ \mathrm{RoPE}(\mathbf{k}, n) \rangle = \mathbf{q}^\top R((n-m)\theta)\,\mathbf{k}$$

Cada par de dimensiones actúa como un canal de frecuencia distinto, codificando relaciones a diferentes escalas de distancia — desde tokens adyacentes hasta dependencias de largo alcance.

RoPE es el esquema de posición estándar en los LLMs open-weight modernos: **LLaMA, Mistral, Gemma, Qwen**, entre otros.